

Documentação Técnica — Sistema de Portaria Inteligente

Grupo Gomes Pires | Uso interno — Equipe de TI

Versão: 1.0 **Data:** 17 de agosto de 2026 **Público-alvo:** desenvolvedores e equipe técnica responsáveis pela construção, manutenção e suporte do sistema

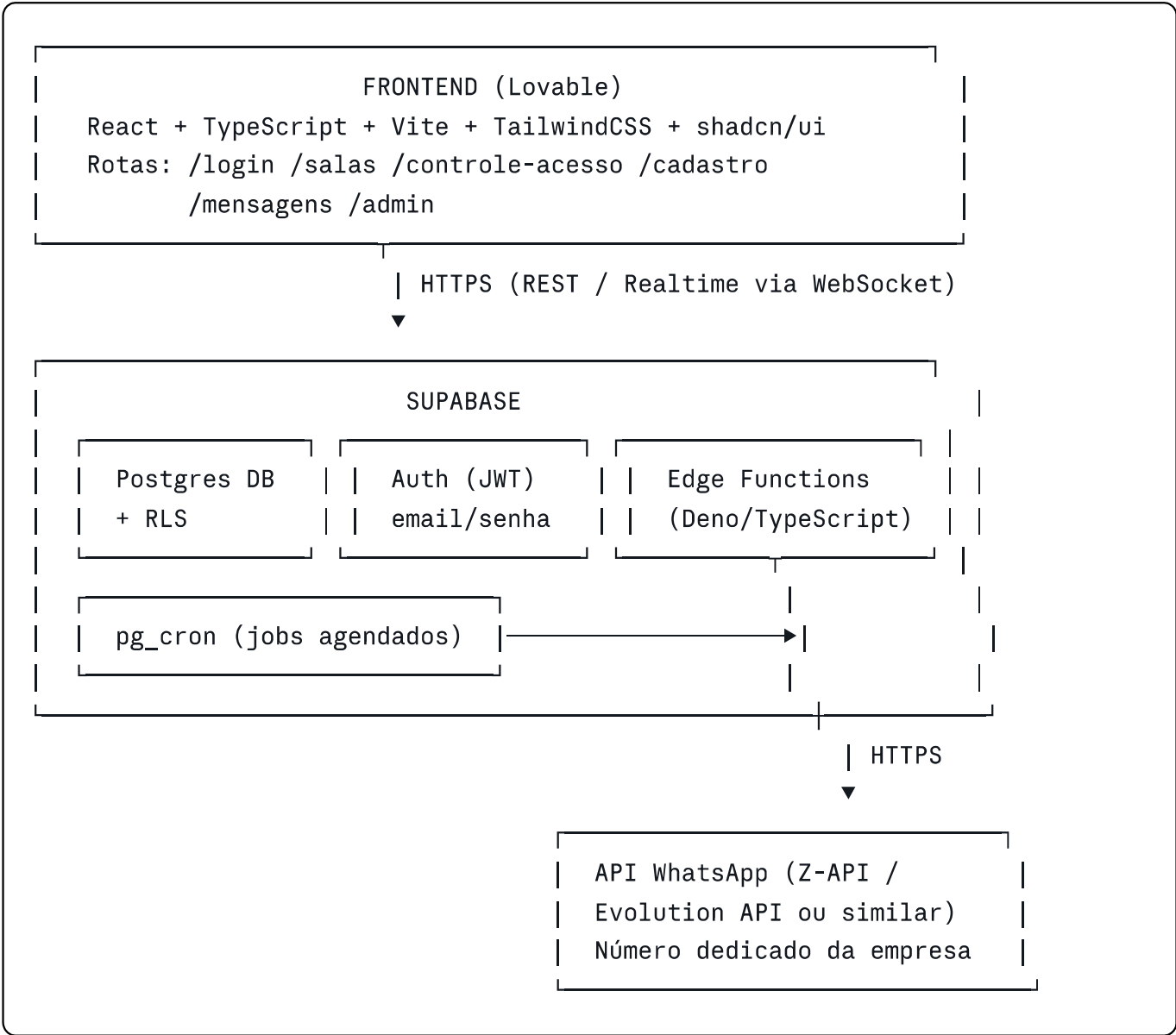
1. Introdução

Este documento descreve a arquitetura técnica, o modelo de dados, os fluxos de negócio e as integrações do novo Sistema de Portaria Inteligente, desenvolvido na plataforma **Lovable** (frontend) com **Supabase** (backend-as-a-service). O objetivo é dar à equipe de TI todo o contexto necessário para dar manutenção, estender funcionalidades e diagnosticar problemas sem depender de conhecimento tácito de quem construiu o sistema.

O documento assume conhecimento básico de desenvolvimento web, SQL e APIs REST.

2. Arquitetura Geral

2.1 Visão de componentes



2.2 Stack tecnológica

Camada	Tecnologia	Observações
Frontend	React + TypeScript + Vite	Gerado e hospedado via Lovable
Estilização	TailwindCSS + shadcn/ui	Componentes prontos, customizados com a paleta da marca
Backend / DB	Supabase (Postgres)	Inclui Auth, Row Level Security (RLS), Storage e Edge Functions
Autenticação	Supabase Auth (email/senha)	Restrição de domínio aplicada em trigger de banco (seção 4.2)
Notificações	Edge Function (Deno) → API externa de WhatsApp	Requer conta ativa em serviço tipo Z-API/Evolution API
Jobs agendados	Supabase (pg_cron) ou Scheduled Edge Function	Usado no lembrete de reunião (20 min antes)
Hospedagem	Lovable Cloud (ou domínio próprio via CNAME)	A definir com o time de infraestrutura

2.3 Por que essa stack

Lovable gera aplicações React que já vêm integradas nativamente ao Supabase — isso cobre banco de dados, autenticação e Edge Functions sem infraestrutura adicional para provisionar. A única peça **fora** desse ecossistema é a integração de WhatsApp, que depende de um serviço externo (ver seção 7).

3. Perfis de Acesso (RBAC)

Perfil	Tabela/valor	Acesso
Admin	system_users.role = 'admin'	Todas as rotas, incluindo /admin
Recepção	system_users.role = 'recepcao'	Todas as rotas exceto /admin

O controle de acesso é feito em duas camadas:

- Frontend:** rotas protegidas por verificação de `role` no contexto de autenticação (esconde/mostra menu e bloqueia navegação direta por URL).
- Backend (obrigatório, não confiar só no frontend):** políticas de **Row Level Security (RLS)** no Postgres, validando o `role` do usuário autenticado a cada operação. Isso impede que alguém manipule requisições diretamente na API e acesse dados/ações fora do seu perfil.

4. Modelo de Dados

4.1 Schema completo (SQL)

sql

```

-- =====
-- CONTAS DE LOGIN DO SISTEMA
-- =====

create table system_users (
  id uuid primary key references auth.users(id) on delete cascade,
  name text not null,
  email text not null unique,
  role text not null check (role in ('admin', 'recepcao')),
  created_at timestamptz default now()
);

-- =====
-- COLABORADORES (para notificação - não fazem login)
-- =====

create table collaborators (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  email text,
  phone text not null,
  cpf text,
  created_at timestamptz default now()
);

-- =====
-- SALAS DE REUNIÃO
-- =====

create table rooms (
  id uuid primary key default gen_random_uuid(),
  name text not null unique,
  created_at timestamptz default now()
);

-- =====
-- AGENDAMENTOS DE SALA
-- =====

create table room_bookings (
  id uuid primary key default gen_random_uuid(),
  room_id uuid not null references rooms(id),
  title text not null,
  booking_date date not null,
  start_time time not null,
  duration_minutes int not null check (duration_minutes > 0),
  end_time time generated always as (start_time + (duration_minutes || ' minute'
  service text check (service in ('com_cafe', 'sem_cafe')) default 'sem_cafe',
  created_by uuid references system_users(id),
  created_at timestamptz default now()

```

```

);

create table booking_participants (
    booking_id uuid references room_bookings(id) on delete cascade,
    collaborator_id uuid references collaborators(id) on delete cascade,
    primary key (booking_id, collaborator_id)
);

-- Índice para checagem rápida de conflito de horário
create index idx_bookings_room_date on room_bookings (room_id, booking_date);

-- =====
-- CONTROLE DE ACESSO (entradas/saídas)
-- =====

create table access_logs (
    id uuid primary key default gen_random_uuid(),
    action text not null check (action in ('entrada', 'saida')),
    category text not null check (category in (
        'entregas_mercadoria', 'atendimento', 'entrevista_rh',
        'prestadores_servico', 'stands', 'treinamentos_onboarding', 'visitas_reunio
    )),
    visitor_name text not null,
    phone text,
    document text,
    observations text,
    notify boolean default false,
    notified_collaborator_id uuid references collaborators(id),
    created_by uuid references system_users(id),
    created_at timestampz default now()
);

create index idx_access_logs_created_at on access_logs (created_at desc);

-- =====
-- TEMPLATES DE MENSAGEM
-- =====

create table message_templates (
    id uuid primary key default gen_random_uuid(),
    category text, -- null para templates não ligados a categoria (lembrete/agenda
    type text not null check (type in ('entrada', 'saida', 'agendamento', 'lembre
    subject text,
    message text not null,
    is_default boolean default true,
    updated_at timestampz default now()
);

-- =====

```

```
-- LOG DE AUDITORIA
-- =====
create table audit_log (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references system_users(id),
  action text not null,
  entity text,
  entity_id uuid,
  created_at timestamptz default now()
);
```

4.2 Trigger — restrição de domínio de e-mail

A validação de domínio (`@grupogomespires.com.br`) **não deve** ficar só no formulário do frontend — precisa ser garantida no banco:

```
sql

create or replace function check_email_domain()
returns trigger as $$
begin
  if new.email !~* '@grupogomespires\.com\.br$' then
    raise exception 'E-mail deve pertencer ao domínio @grupogomespires.com.br';
  end if;
  return new;
end;
$$ language plpgsql;

create trigger trg_check_email_domain
  before insert or update on system_users
  for each row execute function check_email_domain();
```

4.3 Políticas de Row Level Security (exemplos)

RLS deve ser **habilitado em todas as tabelas**. Exemplos representativos:

```
sql
```

```

alter table rooms enable row level security;
alter table system_users enable row level security;
alter table audit_log enable row level security;

-- Qualquer usuário autenticado (admin ou recepção) pode LER salas
create policy "Usuários autenticados podem ver salas"
  on rooms for select
  using (auth.role() = 'authenticated');

-- Só Admin pode criar/editar/excluir salas
create policy "Somente admin gerencia salas"
  on rooms for all
  using (
    exists (
      select 1 from system_users
      where id = auth.uid() and role = 'admin'
    )
  );

-- Log de auditoria: só admin pode ler; ninguém edita/exclui via API
create policy "Somente admin lê auditoria"
  on audit_log for select
  using (
    exists (
      select 1 from system_users
      where id = auth.uid() and role = 'admin'
    )
  );

```

Ponto de atenção para a equipe de TI: repetir esse padrão de policy para `system_users` (CRUD restrito a admin), `access_logs` e `collaborators` (CRUD liberado para qualquer autenticado, conforme regra de negócio confirmada) e `room_bookings` (edição/exclusão liberada para qualquer autenticado, sem restrição por criador — regra de negócio confirmada com a operação).

5. Estrutura de Rotas (Frontend)

Rota	Tela	Acesso
/login	Autenticação	Público
/salas	Dashboard de Salas de Reunião	Admin, Recepção
/controle-acesso	Registro e histórico de acessos	Admin, Recepção
/cadastro	Gestão de colaboradores	Admin, Recepção
/mensagens	Edição de templates de notificação	Admin, Recepção
/admin	Painel administrativo (5 submódulos)	Somente Admin
/admin/salas	CRUD de salas	Somente Admin
/admin/usuarios	CRUD de contas de login	Somente Admin
/admin/relatorios	Métricas e exportação	Somente Admin
/admin/whatsapp	Status/configuração da integração	Somente Admin
/admin/auditoria	Log de ações	Somente Admin

Todas as rotas exceto /login devem usar um **layout wrapper autenticado** que: (1) redireciona para /login se não houver sessão válida; (2) redireciona /admin/* para /salas se role !== 'admin'.

6. Fluxos de Negócio Críticos

6.1 Registro de Controle de Acesso com notificação

1. Recepção preenche formulário (tipo de ação, categoria, nome, telefone, documento, observações)
2. [opcional] Marca "Enviar Aviso" → seleciona colaborador da lista (tabela collaborators)
3. Frontend chama insert em access_logs
4. Trigger (ou chamada explícita do frontend) invoca Edge Function "notify-access" passando: category, action, visitor_name, collaborator_id
5. Edge Function:
 - a. Busca o template correspondente em message_templates (category + type = action)
 - b. Interpola variáveis: [NOME_COLABORADOR], [NOME_VISITANTE], [ACAO]
 - c. Busca telefone do colaborador em collaborators
 - d. Faz POST para a API do serviço de WhatsApp (seção 7)
6. Resposta da API é logada (sucesso/erro) – recomenda-se salvar em uma coluna ou tabela de log de envio para diagnóstico

6.2 Agendamento de sala com notificações automáticas

1. Recepção preenche "Novo Agendamento" (sala, título, data, horário, duração, serviço, participantes)
2. Frontend valida conflito de horário ANTES de submeter:
SELECT existe agendamento na mesma room_id, mesma booking_date, com intervalo [start_time, end_time] sobreposto ao novo
3. Se não houver conflito: insert em room_bookings + inserts em booking_participants
4. Disparo automático (sem checkbox, ao contrário do fluxo 6.1):
Edge Function "notify-booking" é chamada para CADA participante, usando o template "Nova Reunião Agendada" (Grupo C)
5. Um job é agendado (via pg_cron ou fila) para disparar o template "Lembrete de Reunião" exatamente 20 minutos antes de start_time

Validação de conflito (exemplo de query):

sql

```
select exists (  
  select 1 from room_bookings  
  where room_id = :room_id  
    and booking_date = :booking_date  
    and (start_time, end_time) overlaps (:new_start_time, :new_end_time)  
) as has_conflict;
```

6.3 Job do Lembrete de Reunião (20 min antes)

Opção recomendada: **Supabase** (`pg_cron`) rodando a cada minuto, verificando agendamentos cujo (`start_time`) está entre 19 e 21 minutos no futuro (janela de tolerância para evitar perda por atraso de execução), e que ainda não tiveram lembrete disparado (necessário um campo booleano (`reminder_sent`) em (`room_bookings`) para controle de idempotência — **adicionar ao schema da seção 4.1**).

```
sql
```

```
alter table room_bookings add column reminder_sent boolean default false;
```

```
sql
```

```
select cron.schedule(
  'lembrete-reuniao',
  '* * * * *', -- a cada minuto
  $$
    select net.http_post(
      url := '<URL_DA_EDGE_FUNCTION>/send-reminders',
      headers := '{"Authorization": "Bearer <SERVICE_ROLE_KEY>"}'::jsonb
    );
  $$
);
```

A Edge Function (`send-reminders`) faz a query de agendamentos elegíveis, dispara as mensagens e marca (`reminder_sent = true`).

7. Integração com WhatsApp

7.1 Visão geral

O sistema original usa um número de WhatsApp dedicado da empresa, conectado via serviço terceirizado (padrão: **Z-API** ou **Evolution API** — conexão por QR code, não é a API oficial da Meta/WhatsApp Business Platform). **A equipe de TI deve confirmar qual serviço está em uso atualmente e obter as credenciais de API antes de iniciar essa parte do desenvolvimento.**

7.2 Onde ficam as credenciais

Nunca armazenar token/API key da integração de WhatsApp no frontend ou em variáveis expostas ao navegador. Devem ficar como **Secrets do Supabase**, acessíveis apenas pelas Edge Functions:

```
bash
```

```
supabase secrets set WHATSAPP_API_URL=https://api.z-api.io/instances/xxx  
supabase secrets set WHATSAPP_API_TOKEN=xxxxxxxxxxxxx
```

7.3 Exemplo de Edge Function (envio de mensagem)

ts

```
// supabase/functions/notify-access/index.ts
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

Deno.serve(async (req) => {
  const { category, action, visitorName, collaboratorId } = await req.json();

  const supabase = createClient(
    Deno.env.get("SUPABASE_URL")!,
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
  );

  const { data: collaborator } = await supabase
    .from("collaborators")
    .select("name, phone")
    .eq("id", collaboratorId)
    .single();

  const { data: template } = await supabase
    .from("message_templates")
    .select("message")
    .eq("category", category)
    .eq("type", action)
    .single();

  const message = template.message
    .replace("[NOME_COLABORADOR]", collaborator.name)
    .replace("[NOME_VISITANTE]", visitorName)
    .replace("[ACAO]", action === "entrada" ? "entrada" : "saída");

  const response = await fetch(
    `${Deno.env.get("WHATSAPP_API_URL")}/send-text`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Client-Token": Deno.env.get("WHATSAPP_API_TOKEN")!,
      },
      body: JSON.stringify({
        phone: collaborator.phone,
        message,
      }),
    }
  );

  const result = await response.json();
```

```
return new Response(JSON.stringify({ ok: response.ok, result }), {
  headers: { "Content-Type": "application/json" },
});
```

O exemplo acima é ilustrativo — o payload exato (`send-text`, `phone`, `message`) varia conforme o provedor real (Z-API, Evolution API, etc.). **Ajustar conforme a documentação do serviço contratado pela empresa.**

7.4 Tratamento de falhas

Recomendações para a equipe de TI:

- Registrar toda tentativa de envio (sucesso/erro) numa tabela `notification_log`, incluindo o payload de resposta da API — essencial para investigar reclamações de "mensagem não chegou".
- Implementar retry com backoff (ex: 3 tentativas) para falhas de rede/timeout.
- Monitorar erros de autenticação da API externa (token expirado, número desconectado) e alertar o Admin (ex: e-mail ou banner no painel `/admin/whatsapp`).

8. Segurança

Item	Prática recomendada
Autenticação	Supabase Auth com senha forte obrigatória; sem opção de auto-cadastro público
Autorização	RLS em 100% das tabelas — nunca liberar acesso via <code>service_role</code> no frontend
Domínio de e-mail	Validado por trigger de banco (seção 4.2), não só no formulário
Secrets	Nunca em <code>.env</code> versionado ou exposto ao bundle do frontend; usar Supabase Secrets
Inputs	Validar/sanitizar telefone, CPF e e-mail tanto no frontend quanto via constraint/trigger no banco
Auditoria	Toda ação de criação/edição/exclusão feita pelas telas administrativas deve gravar em <code>audit_log</code>
LGPD	O sistema armazena CPF, telefone e nome de visitantes/candidatos — validar com jurídico/compliance a necessidade de política de retenção e descarte desses dados

9. Ambientes e Deploy

- **Desenvolvimento:** projeto Supabase de desenvolvimento separado do de produção, para não afetar dados reais durante testes.

- **Produção:** publicação via Lovable, com domínio próprio da empresa configurado (CNAME).
 - **Migrações de banco:** versionar todas as alterações de schema (SQL) em um diretório `supabase/migrations`, mesmo trabalhando primariamente pela interface do Lovable — evita perda de rastreabilidade.
 - **Backup:** confirmar periodicidade de backup automático do Supabase (padrão do plano contratado) e, se necessário, configurar backup adicional.
-

10. Roadmap Técnico Sugerido

Alinhado às fases já apresentadas à gerência (documento de proposta), a ordem de implementação recomendada é:

1. Schema de banco + RLS + autenticação
 2. Layout base + rotas protegidas
 3. Módulo Salas de Reunião (CRUD de agendamento + validação de conflito)
 4. Módulo Controle de Acesso
 5. Módulo Cadastro de Colaboradores (+ importação CSV/XLSX)
 6. Módulo Mensagens Automáticas (CRUD de templates)
 7. Integração WhatsApp (Edge Functions + credenciais reais)
 8. Job de lembrete de reunião (pg_cron)
 9. Painel Administrativo (5 submódulos)
 10. Testes de carga leve, revisão de RLS, homologação com a recepção
-

11. Glossário

Termo	Significado
RLS	Row Level Security — controle de acesso a linhas do banco por política SQL
Edge Function	Função serverless (Deno) hospedada no Supabase, usada para lógica de backend
pg_cron	Extensão do Postgres para agendar execução periódica de tarefas via SQL
JWT	Token usado pelo Supabase Auth para autenticar requisições do usuário logado
RBAC	Role-Based Access Control — controle de acesso baseado em papel/perfil do usuário

12. Pendências técnicas a resolver com a empresa

- Confirmar qual serviço de API de WhatsApp está em uso (Z-API, Evolution API ou outro) e obter credenciais de acesso

- Confirmar plano do Supabase (limites de Edge Functions, banco, backup) necessário para o volume de uso da empresa
- Definir política de retenção de dados de visitantes (LGPD) junto ao jurídico
- Confirmar domínio próprio a ser usado em produção